Министерство науки и высшего образования Российской Федерации
Пермский филиал федерального государственного бюджетного
образовательного учреждения высшего образования
«Российский экономический университет
имени Г. В. Плеханова»

СОПРОВОДИТЕЛЬНАЯ ЗАПИСКА

к практической работе № 2

на тему

«Разработка калькулятора на C# и WPF»

Выполнил:
студент группы ИПс-21
Ашлапов Иван

Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 Постановка задачи.....4
1.1 Средства разработки.....4
2 Структура решения.....5
3 Создание решения в Visual Studio.....6
3.1 Добавление ссылок на проекты.....6
3.2 Сборка и запуск приложения.....6
4 Разработка библиотеки классов.....7
4.1 Класс Calculator.....7
4.2 Класс ExpressionCalculator.....7
5 Разработка интерфейса WPF.....8
5.1 Разметка окна.....8
5.2 Модель представления и команды.....8
5.3 Ввод с клавиатуры.....9
5.4 История вычислений.....9
5.5 Продолжение вычислений.....9
5.6 Сообщения об ошибках.....10
5.7 Проверка элементов интерфейса.....10
6 Модульное тестирование.....11
6.1 Запуск и результаты тестирования.....11
7 Проверка работы программы.....12
7.1 Проверка обработки ошибок.....13
8 Подготовка проекта к размещению на GitHub.....14
ЗАКЛЮЧЕНИЕ.....15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....16
ПРИЛОЖЕНИЕ А Класс Calculator.....17
ПРИЛОЖЕНИЕ Б Класс ExpressionCalculator.....19

ВВЕДЕНИЕ

Практическая работа посвящена созданию калькулятора на языке C# с графическим интерфейсом WPF. Для разделения обязанностей вычислительная часть оформлена как библиотека классов, а её поведение проверяется отдельным проектом модульных тестов. Разработанное решение получило имя AshlapovCalculator.

1 Постановка задачи

Программа должна выполнять сложение, вычитание, умножение, деление и возведение в степень. Выражения вводятся кнопками или с клавиатуры. При вычислении учитываются приоритет операций, вложенные скобки, дробные числа и отрицательные значения.

Основные возможности программы приведены в таблице 1.

Таблица 1 — Основные возможности калькулятора

Возможность	Пример и результат
Приоритет умножения	2+3*4 = 14
Скобки	(2+3)*4 = 20
Степень справа налево	2^3^2 = 512
Минус перед степенью	−2^2 = −4; (−2)^2 = 4
Отрицательная степень	2^−2 = 0,25
Дробные числа	0,1+0,2 = 0,3
Обработка ошибок	5/0 — сообщение о делении на ноль

Для удобства повторных расчётов я предусмотрел историю последних 20 успешных вычислений. После вывода результата пользователь может продолжить расчёт, введя оператор, либо начать новое выражение с цифры. При этом все пять арифметических операций доступны как с помощью кнопок, так и с клавиатуры.

1.1 Средства разработки

C#, .NET 8, WPF, XAML и MSTest 4.0.2. Приложение рассчитано на Windows. Основная логика и модульные тесты имеют целевую платформу net8.0; интерфейс — net8.0-windows.

2 Структура решения

Назначение каждого проекта решения указано в таблице 2.

Таблица 2 — Проекты решения AshlapovCalculator

Проект	Назначение
AshlapovCalculator.Core	Библиотека арифметики и разбора выражений
AshlapovCalculator.Wpf	Окно, стили, модель представления и команды
AshlapovCalculator.Tests	Модульные тесты библиотеки и модели представления

3 Создание решения в Visual Studio

В Visual Studio я создал проект по шаблону «Приложение WPF» для языка C#. В качестве имени проекта указано AshlapovCalculator.Wpf, а в качестве имени решения — AshlapovCalculator. Для приложения выбрана целевая платформа .NET 8.

Следующим этапом стало добавление библиотеки AshlapovCalculator.Core через контекстное меню решения. Вместо созданного по шаблону файла Class1.cs в библиотеку включены классы Calculator и ExpressionCalculator. Первый отвечает за отдельные операции, второй — за вычисление выражения целиком.

Для автоматической проверки создан проект AshlapovCalculator.Tests, в который установлен пакет MSTest 4.0.2. Приложение и тесты подключены к библиотеке Core ссылками на проект. В настройках решения запускаемым выбран AshlapovCalculator.Wpf.

3.1 Добавление ссылок на проекты

Листинг 1 — Фрагмент программы

```
AshlapovCalculator.Wpf   -> AshlapovCalculator.Core
AshlapovCalculator.Tests -> AshlapovCalculator.Core
```

Модель представления и RelayCommand не зависят от визуальных классов WPF. Их исходные файлы подключены к проекту тестов как ссылки Compile. Поэтому поведение интерфейса проверяется без создания окна и без изменения целевой платформы тестов.

3.2 Сборка и запуск приложения

Открыть AshlapovCalculator.sln, дождаться восстановления NuGet, назначить проект Wpf запускаемым и нажать F5. Сборка из консоли: dotnet build AshlapovCalculator.sln -c Release.

4 Разработка библиотеки классов

4.1 Класс Calculator

Класс Calculator содержит методы Add, Subtract, Multiply, Divide и Power. Метод Calculate выбирает действие через switch. Степень вычисляется с помощью Math.Pow. Неизвестный оператор приводит к ArgumentException.

Листинг 2 — Обработка деления на ноль

```
public double Divide(double a, double b)
{
    if (b == 0)
        throw new DivideByZeroException("Деление на ноль
невозможно");
    return a / b;
}
```

4.2 Класс ExpressionCalculator

SplitExpression разбирает строку на три списка: числа, их унарные знаки и операции. Ввод с запятой и точкой обрабатывается одинаково. При встрече открывающей скобки FindClosingBracket находит парную закрывающую скобку с учётом вложенности; внутреннее выражение вычисляется рекурсивно.

Сначала CalculatePowers выполняет степени справа налево. Затем ApplySigns применяет унарные минусы; после этого CalculateOperations обрабатывает умножение и деление, а затем сложение и вычитание. Такой порядок различает −2^2 и (−2)^2.

Листинг 3 — Примеры порядка вычислений

```
2+3*4:  [2, 3, 4], [+, *] -> [2, 12], [+] -> 14
2^3^2:  2^(3^2) = 2^9 = 512
-2^2:   -(2^2) = -4
2^-2:   1/(2^2) = 0,25
```

Ошибки ввода и неверные скобки вызывают исключения. Значения NaN и бесконечность отклоняются. Результат округляется до 10 знаков после запятой, поэтому 0,1+0,2 = 0,3.

5 Разработка интерфейса WPF

5.1 Разметка окна

В MainWindow.xaml расположены заголовок, дисплей, сетка клавиш и правая колонка истории. Светлый фон сочетается с тёмным зелёным дисплеем. Операторы выделены мягким зелёным цветом, а клавиша «=» — насыщенным. Стили кнопок находятся в App.xaml.

Панель ввода содержит десять цифр, десятичный разделитель, пять операторов, круглые скобки и служебные клавиши. Кнопка C очищает выражение, а клавиша удаления убирает последний введённый символ. Для умножения и деления на кнопках используются знаки × и ÷, которым в вычислительной части соответствуют символы * и /.

5.2 Модель представления и команды

MainViewModel реализует INotifyPropertyChanged. Свойства Expression, Result и ErrorMessage связаны с текстом дисплея. Команды AddSymbolCommand, CalculateCommand, ClearCommand и BackspaceCommand связывают действия кнопок с методами модели.

Листинг 4 — Привязка кнопки к команде

```
<Button Content="7" Style="{StaticResource CalcButton}"
        Command="{Binding AddSymbolCommand}"
CommandParameter="7" />
```

RelayCommand реализует ICommand и вызывает переданное действие. История хранится в ObservableCollection, поэтому добавление или очистка записей сразу отражается в интерфейсе.

5.3 Ввод с клавиатуры

Клавиши управления и соответствующие действия перечислены в таблице 3.

Таблица 3 — Управление с клавиатуры

Клавиша	Действие
0–9, +, −, *, /, ^, (,)	Добавить цифру или оператор
, или .	Добавить десятичный разделитель
Enter или =	Вычислить выражение
Backspace	Удалить последний символ
Esc	Очистить выражение и результат

5.4 История вычислений

После успешного расчёта выражение и результат добавляются в начало истории. В ней сохраняется не более 20 записей; самые старые удаляются автоматически. Повторное нажатие «=» для того же выражения не дублирует последнюю запись. Ошибочные выражения в историю не попадают.

Нажатие на запись возвращает выражение на дисплей для редактирования и повторного расчёта. Клавиша C очищает текущий ввод, сохраняя историю. Отдельная кнопка «Очистить» в колонке истории удаляет только записи. После закрытия приложения история не сохраняется.

5.5 Продолжение вычислений

После 2+3 = 5 нажатие * и затем 4 формирует 5*4 и даёт 20. Если вместо оператора ввести цифру 7, начинается новое выражение 7. Десятичный разделитель после результата начинает новую дробь с нуля.

При продолжении отрицательного результата он заключается в скобки: после 2−4 = −2 ввод ^2 вычисляет (−2)^2 = 4. Для передачи результата в парсер используется запись без экспоненты и с независимым от локали десятичным разделителем.

5.6 Сообщения об ошибках

На дисплее выводится короткое слово «Ошибка», а причина размещается отдельной строкой с переносом текста. Это сохраняет читаемость длинного сообщения. Выражение остаётся доступным для исправления с помощью Backspace либо полной очистки.

5.7 Проверка элементов интерфейса

Проверены начальное состояние, заполненная история, ошибка деления на ноль и минимальный размер окна. Кнопки доступны через Tab и имеют выделение фокуса. При проверке окна ошибки привязок данных WPF не обнаружены.

6 Модульное тестирование

Все тесты запускаются MSTest. Сценарии с несколькими наборами входных значений оформлены через [DataRow]. Тесты сравнивают результаты операций и типы исключений. Всего в проекте 47 проверок.

Состав модульных тестов представлен в таблице 4.

Таблица 4 — Состав модульных тестов

Класс тестов	Число	Проверяемое поведение
CalculatorTests	8	Пять операций, отрицательная степень, деление на ноль и неизвестная операция
ExpressionCalculatorTests	22	Приоритет, скобки, унарный минус, цепочка степеней, дроби и ошибки выражений
MainViewModelTests	17	Новый ввод, точность продолжения результата, история, очистка и исправление ошибок

Листинг 5 — Фрагмент программы

```
[TestMethod]
public void Divide_ByZero_ThrowsException()
{
    Assert.ThrowsExactly<DivideByZeroException>(
        () => _calculator.Divide(5, 0));
}
```

6.1 Запуск и результаты тестирования

Листинг 6 — Результаты запуска модульных тестов

```
dotnet test AshlapovCalculator.sln -c Release

Всего тестов: 47
Пройдено: 47
Не пройдено: 0
Пропущено: 0
```

Проверка выполнена 22.10.2026. Дополнительно решение собрано в конфигурации Release без ошибок и предупреждений. Для проверки кнопок выполнена последовательность ввода 2+3*4 и подтверждён результат 14; обработчик Esc очищает дисплей.

7 Проверка работы программы

В истории видны независимые проверки приоритета операций, скобок, цепочки степеней и дробных чисел. Текущее выражение 0,1+0,2 даёт результат 0,3.

Результат проверки представлен на рисунке 1.

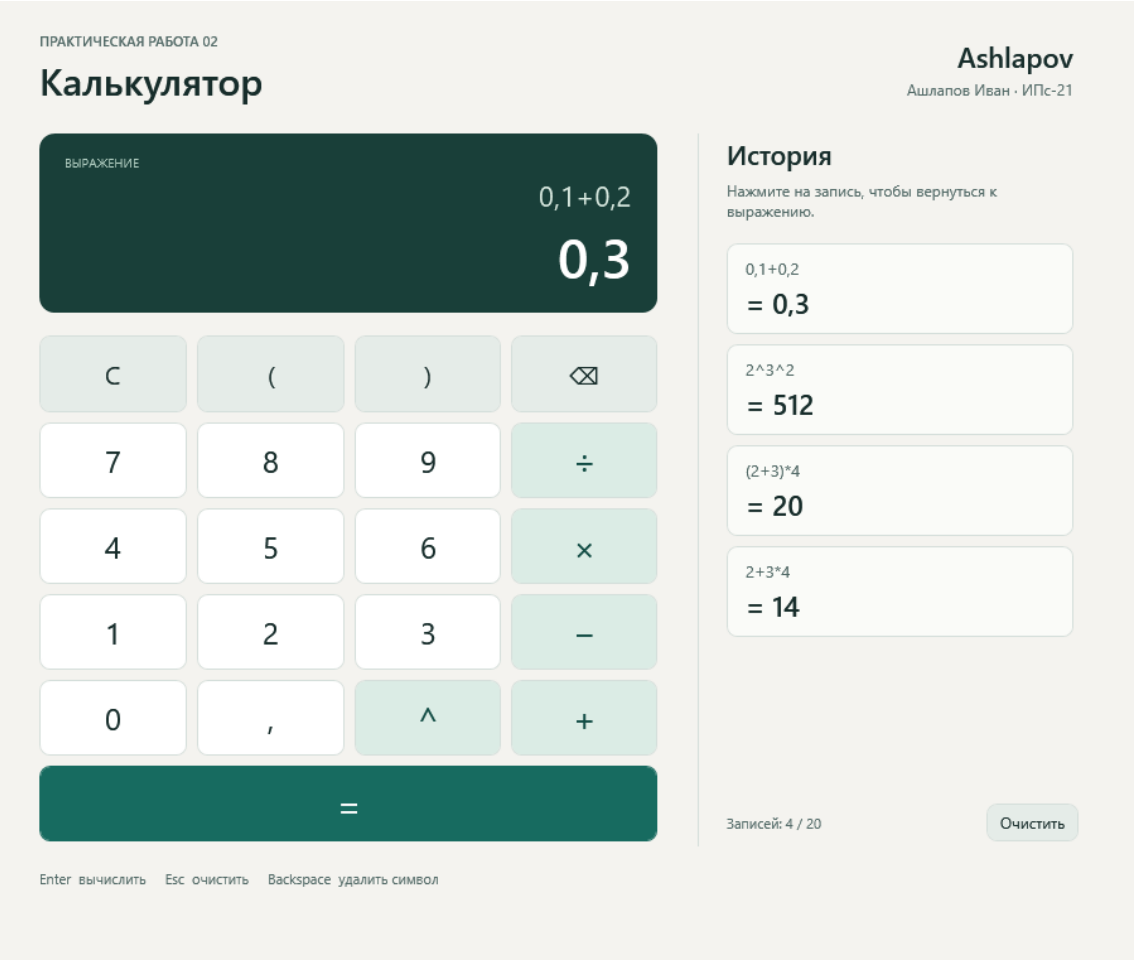


Рисунок 1 — Интерфейс AshlapovCalculator с историей вычислений

Справа новые вычисления располагаются выше предыдущих. Текст автора и группа указаны в заголовке окна и в верхней части интерфейса.

7.1 Проверка обработки ошибок

Попытка вычислить 5/0 приводит к сообщению «Деление на ноль невозможно». История сохраняет предыдущие успешные вычисления, а ошибочная запись в неё не добавляется.

Результат проверки представлен на рисунке 2.

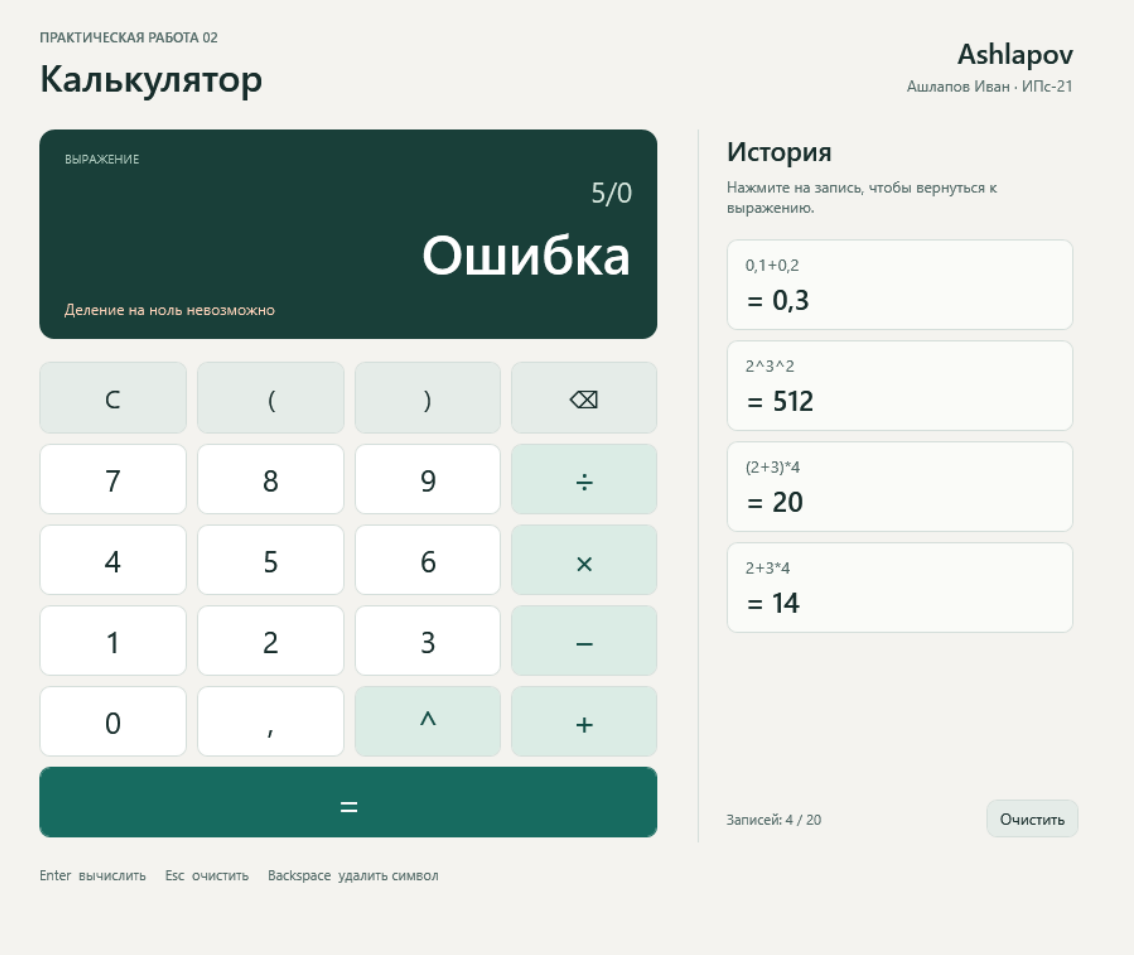


Рисунок 2 — Сообщение об ошибке деления на ноль

Для исправления можно удалить последний символ клавишей Backspace и ввести другой делитель. Например, после замены 0 на 2 выражение 5/2 даёт 2,5. Esc или C полностью очищает текущий ввод.

8 Подготовка проекта к размещению на GitHub

Для передачи работы можно создать репозиторий AshlapovCalculator и загрузить содержимое папки practice 2 Ashlapov. В корне должны находиться файл решения, README и три папки проектов. Записка в форматах DOCX и PDF находится в docs.

Каталоги bin, obj, .vs и TestResults не включаются в репозиторий. Инструкция по запуску и загрузке приложена в HTML. Перед отправкой работы необходимо загрузить исходные файлы и записку в репозиторий, затем проверить доступность ссылки для преподавателя.

ЗАКЛЮЧЕНИЕ

В результате практической работы я разработал калькулятор с интерфейсом WPF и отдельной библиотекой вычислений. Программа обрабатывает пять арифметических операций, учитывает их приоритет, поддерживает вложенные скобки, дробные и отрицательные числа. Предусмотрены ввод с клавиатуры, история вычислений и продолжение расчёта с полученного результата. Все 47 модульных тестов завершились успешно. Разделение решения на проекты позволяет проверять вычислительную часть независимо от окна приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 WPF. Документация и исходный код Microsoft — Текст : электронный. — URL: https://github.com/dotnet/wpf (дата обращения: 22.10.2026).

2 Microsoft Test Framework. Документация MSTest — Текст : электронный. — URL: https://github.com/microsoft/testfx (дата обращения: 22.10.2026).

3 Microsoft Learn. Модульное тестирование C# с помощью MSTest и .NET — Текст : электронный. — URL: https://learn.microsoft.com/dotnet/core/testing/unit-testing-with-mstest (дата обращения: 22.10.2026).

4 Документация GitHub — Текст : электронный. — URL: https://docs.github.com/ru (дата обращения: 22.10.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Класс Calculator

AshlapovCalculator.Core\Calculator.cs

```
namespace AshlapovCalculator.Core;

/// <summary>Арифметические операции калькулятора.</summary>
public class Calculator
{
    public double Add(double a, double b)
    {
        return a + b;
    }

    public double Subtract(double a, double b)
    {
        return a - b;
    }

    public double Multiply(double a, double b)
    {
        return a * b;
    }

    public double Divide(double a, double b)
    {
        if (b == 0)
        {
            throw new DivideByZeroException("Деление на ноль
невозможно");
        }

        return a / b;
    }

    public double Power(double a, double b)
    {
        return Math.Pow(a, b);
    }

    public double Calculate(double a, char operation, double b)
    {
        switch (operation)
        {
            case '+':
                return Add(a, b);
```

```
            case '-':
                return Subtract(a, b);
            case '*':
                return Multiply(a, b);
            case '/':
                return Divide(a, b);
            case '^':
                return Power(a, b);
            default:
                throw new ArgumentException("Неизвестная
операция: " + operation);
        }
    }
}
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Класс ExpressionCalculator

AshlapovCalculator.Core\ExpressionCalculator.cs

```
using System.Globalization;

namespace AshlapovCalculator.Core;

/// <summary>Вычисляет выражение вида "2+3*4" или "(2+3)*4" с
учётом приоритета операций и скобок.</summary>
public class ExpressionCalculator
{
    private const string Operations = "+-*/^";

    private readonly Calculator _calculator = new Calculator();

    public double Calculate(string expression)
    {
        if (string.IsNullOrWhiteSpace(expression))
        {
            throw new ArgumentException("Выражение не введено");
        }

        var numbers = new List<double>();
        var signs = new List<int>();
        var operations = new List<char>();
        SplitExpression(expression.Replace(" ", ""), numbers,
signs, operations);

        // Сначала степень, потом минус перед числом, потом
умножение и деление, потом сложение и вычитание.
        CalculatePowers(numbers, signs, operations);
        ApplySigns(numbers, signs);
        CalculateOperations(numbers, operations, "*/");
        CalculateOperations(numbers, operations, "+-");

        double result = numbers[0];
        if (double.IsNaN(result) || double.IsInfinity(result))
        {
            throw new ArithmeticException("Результат невозможно
вычислить");
        }

        // Округление убирает погрешность вида 0,1 + 0,2 =
0,30000000000000004.
```

```
        return Math.Round(result, 10);
    }

    // Делит выражение на числа, их знаки и операции: "-2+3*4"
-> [2, 3, 4], [-1, 1, 1] и ['+', '*'].
    private void SplitExpression(string expression, List<double>
numbers, List<int> signs, List<char> operations)
    {
        string number = "";
        int sign = 1;

        for (int i = 0; i < expression.Length; i++)
        {
            char c = expression[i];

            if (char.IsDigit(c) || c == ',' || c == '.')
            {
                number += c;
            }
            else if (c == '(')
            {
                if (number != "")
                {
                    throw new FormatException("Перед скобкой нет
знака операции");
                }

                int end = FindClosingBracket(expression, i);
                string inside = expression.Substring(i + 1, end
- i - 1);
                if (inside == "")
                {
                    throw new FormatException("Пустые скобки");
                }

                // Выражение в скобках вычисляем отдельно и
дальше работаем с ним как с обычным числом.
                number =
Calculate(inside).ToString(CultureInfo.InvariantCulture);
                i = end;

                if (i + 1 < expression.Length && !
Operations.Contains(expression[i + 1]))
                {
                    throw new FormatException("После скобки нет
знака операции");
                }
            }
            else if (c == ')')
            {
                throw new FormatException("Лишняя закрывающая
скобка");
```

```
            }
            else if (c == '-' && number == "" && (i == 0 ||
Operations.Contains(expression[i - 1])))
            {
                // Минус в начале или сразу после знака операции
— это минус перед числом: -5, 2*-3.
                sign = -sign;
            }
            else if (Operations.Contains(c))
            {
                numbers.Add(ParseNumber(number));
                signs.Add(sign);
                operations.Add(c);
                number = "";
                sign = 1;
            }
            else
            {
                throw new FormatException("Недопустимый символ:
" + c);
            }
        }

        numbers.Add(ParseNumber(number));
        signs.Add(sign);
    }

    private static int FindClosingBracket(string expression, int
openIndex)
    {
        int depth = 0;

        for (int i = openIndex; i < expression.Length; i++)
        {
            if (expression[i] == '(')
            {
                depth++;
            }
            else if (expression[i] == ')')
            {
                depth--;
                if (depth == 0)
                {
                    return i;
                }
            }
        }

        throw new FormatException("Не хватает закрывающей
скобки");
    }
```

```
    private static double ParseNumber(string text)
    {
        text = text.Replace(',', '.');

        if (!double.TryParse(text, NumberStyles.Float,
CultureInfo.InvariantCulture, out double value))
        {
            throw new FormatException("Некорректное выражение");
        }

        return value;
    }

    // Степень считается справа налево: 2^3^2 = 2^9.
    // Минус перед основанием применяется после степени (-2^2 =
-4),
    // а минус перед показателем — сразу (2^-2 = 0,25).
    private void CalculatePowers(List<double> numbers, List<int>
signs, List<char> operations)
    {
        for (int i = operations.Count - 1; i >= 0; i--)
        {
            if (operations[i] == '^')
            {
                double exponent = signs[i + 1] * numbers[i + 1];
                numbers[i] = _calculator.Power(numbers[i],
exponent);
                numbers.RemoveAt(i + 1);
                signs.RemoveAt(i + 1);
                operations.RemoveAt(i);
            }
        }
    }

    private static void ApplySigns(List<double> numbers,
List<int> signs)
    {
        for (int i = 0; i < numbers.Count; i++)
        {
            numbers[i] = signs[i] * numbers[i];
        }
    }

    private void CalculateOperations(List<double> numbers,
List<char> operations, string symbols)
    {
        int i = 0;
        while (i < operations.Count)
        {
            if (symbols.Contains(operations[i]))
            {
                numbers[i] = _calculator.Calculate(numbers[i],
```

```
operations[i], numbers[i + 1]);
                numbers.RemoveAt(i + 1);
                operations.RemoveAt(i);
            }
            else
            {
                i++;
            }
        }
    }
}
```